

The `<toolbar>` called *navigator-toolbox* specifies the *merge point* within the browser window that we want to attach to. This specifies a new toolbar just below the address bar in the browser. It lets you edit this section as an HTML document, allowing you to add custom widgets such as label, textbox, buttons, etc. We will also add our *logic.js* file as source reference, where we will write the program logic for our add-on.

Using the above code, we will be creating an XUL based UI for interacting with our add-on. This XUL interface consists of a Label, a textbox (to enter a query) and a button (to trigger the function).

Creating a Chrome manifest for the add-on

Open the file named *chrome.manifest* that we created in our add-on directory (*DDG_add-on*) and then add the following code to it. This tells Firefox to merge our *ui.xul* into browser. *xul* when *browser.xul* loads.

```
content DDG_add-on content/
overlay chrome://browser/content/browser.xul chrome://DDG_add-on/content/ui.xul
```

Copy this code and remember that it must be in a single line. Finally, add the logic behind our add-on.

Adding JavaScript

Open the *logic.js* file located at our content directory in our add-on directory, and add the following code:

```
function DuckDuckGoSearch(event){
    var query = document.getElementById("DDGQuery").value;
    window._content.document.location = "http://www.duckduckgo.com/?q=" + encodeURIComponent(query);
}
```

The above function gets the value of the element with the identity as *DDGQuery*, which we declared in our *ui.xul* file, and stores it in a variable named *query*. Now, using DOM, just append the extracted value with the URL of the DuckDuckGo search engine to load the page with the query entered by the user.

Now, let us make our add-on ready for deployment.

Creating a package for our add-on

Add-ons are generally packaged in zip files, with an XPI (Cross-Platform Installer) file extension. So, zip up all the files of the add-on folder. (**Note:** Zip the files in the *DDG_add-on* folder, in our case: *content* folder, *install.rdf* and *chrome.manifest*. When you open the zip file, you must see these files.) You can do this easily in Windows 7; just select all the files and sub-folders from our add-on (*DDG_add-on*) folder, right click and select *Send To -> "Compressed (Zipped) Folder"*. A *.zip* file, *DDG_addon.zip*, will be



Figure 1: Our add-on running in the browser

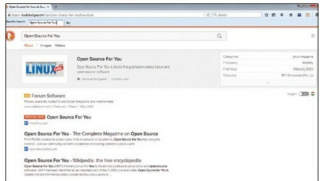


Figure 2: Add-on providing results for the entered text

created. Just rename it as *DDG_addon.xpi* and you are ready to run your first Mozilla Firefox add-on.

Running the add-on

Open the Mozilla Firefox browser and just drag the add-on file into the browser. When you do this, the add-on installer will open up; select the add-on and click *Install*. After installation, restart the Firefox browser.

Now, you will see our add-on's toolbar under the address bar in the browser, as shown in Figure 1.

You can test the add-on by entering any text into the text field provided under the address bar and clicking the *Go* button. For example, we tested the add-on by entering the text 'Open Source For You', and clicked the *Go* button. DuckDuckGo search engine results for 'Open Source For You' were displayed in the browser, as shown in Figure 2. **END** 

References

- [1] <http://en.wikipedia.org/wiki/Mozilla>
- [2] <http://en.wikipedia.org/wiki/Firefox>
- [3] <https://developer.mozilla.org/en-US/docs/Mozilla/Tech/XUL>
- [4] https://developer.mozilla.org/en-US/Add-ons/Overlay-Extensions/XUL_School/Getting_Started_with_Firefox_Extensions

By: Aniket Eknath Kudale

The author has over one year of experience in developing Mozilla Firefox add-ons. He is currently a junior software engineer at Tibco Software Inc., Pune. He can be reached at kudale@aniket.co